2-adic Numbers in Cryptography

From carries to stream ciphers

Siddharth K

May 2026

The main idea

A long bit string can be treated as one 2-adic number:

$$a_0, a_1, a_2, \ldots \quad \longleftrightarrow \quad \alpha = a_0 + a_1 2 + a_2 2^2 + \cdots \in \mathbb{Z}_2.$$

Today I will explain:

1. why this is natural for cryptography,
2. what a feedback with carry shift register is,
3. why its output is a rational 2-adic number.

Why cryptography needs bit streams

Alice has the message written as bits:

$$P_0, P_1, P_2, \ldots, \quad P_i \in \{0, 1\}.$$

A stream cipher uses a keystream:

$$K_0, K_1, K_2, \ldots$$

Alice sends

$$C_i = P_i + K_i \pmod 2.$$

Bob decrypts by doing the same operation:

$$P_i = C_i + K_i \pmod 2.$$

The problem

Alice and Bob need the same keystream, but Eve should not be able to predict it.

2-adic integers

A 2-adic integer is an infinite binary sum

$$a_0 + a_1 2 + a_2 2^2 + a_3 2^3 + \cdots, \qquad a_i \in \{0, 1\}.$$

The key difference from real numbers:

$$2^n \to 0 \quad \text{in the 2-adic metric.}$$

So infinite sums of powers of 2 can converge in $\mathbb{Z}_2$.

Two examples

First example:

$$111111\cdots = 1 + 2 + 2^2 + 2^3 + \cdots = -1 \quad \text{in } \mathbb{Z}_2.$$

Second example:

$$\begin{aligned} 101010\cdots &= 1 + 2^2 + 2^4 + \cdots \\ &= 1 + 4 + 4^2 + \cdots = \frac{1}{1-4} = -\frac{1}{3}. \end{aligned}$$

Takeaway

Repeating bit strings often become rational 2-adic numbers.

Ordinary shift registers

A linear feedback shift register, or LFSR, creates new bits using a linear rule.
Example:

$$a_{n+3} = a_{n+1} + a_n \pmod 2.$$

Why people like LFSRs:

- they are fast,
- they are simple,
- they can make long periodic sequences.

Problem

They are linear over $\mathbb{F}_2$. With enough output, an attacker may recover the rule.

Feedback with carry shift registers

An FCSR is like an LFSR, but it remembers carries.
Choose tap values $q_1, \ldots, q_r \in \{0, 1\}$ and a memory value m_{n-1}. At step n, compute

$$\sigma_n = q_1 a_{n-1} + q_2 a_{n-2} + \cdots + q_r a_{n-r} + m_{n-1}.$$

Then set

$$a_n \equiv \sigma_n \pmod 2, \qquad m_n = \frac{\sigma_n - a_n}{2}.$$

Simple picture

LFSR: $1 + 1 = 0$ mod 2, and the extra 1 disappears.
FCSR: $1 + 1 = 10_2$, so the output is 0 and the carry is 1.

The connection integer

The taps define a number called the connection integer:

$$q = -1 + q_1 2 + q_2 2^2 + \cdots + q_r 2^r.$$

Usually we take $q > 0$, so q is odd.

Main theorem

Let

$$\alpha = a_0 + a_1 2 + a_2 2^2 + \cdots$$

be the 2-adic number made from the output of an FCSR with connection integer q. Then there is an integer p such that

$$\alpha = \frac{p}{q} \quad \text{in } \mathbb{Z}_2.$$

Proof idea

Start with

$$\alpha = x + \sum_{n=r}^{\infty} a_n 2^n, \qquad x = a_0 + a_1 2 + \cdots + a_{r-1} 2^{r-1}.$$

From the FCSR rule,

$$a_n = q_1 a_{n-1} + \cdots + q_r a_{n-r} + m_{n-1} - 2m_n.$$

Substitute this into the tail:

$$\alpha = x + \sum_{i=1}^{r} q_i \sum_{n=r}^{\infty} a_{n-i} 2^n + \sum_{n=r}^{\infty} m_{n-1} 2^n - \sum_{n=r}^{\infty} m_n 2^{n+1}.$$

Proof idea

For the bit sums, re-index:

$$\sum_{n=r}^{\infty} a_{n-i} 2^n = 2^i \left(\alpha - X_i\right),$$

where

$$X_i = \sum_{j=0}^{r-i-1} a_j 2^j$$

is just a finite starting piece.

For the memory sums, take finite sums first:

$$\sum_{n=r}^{N} m_{n-1} 2^n - \sum_{n=r}^{N} m_n 2^{n+1} = m_{r-1} 2^r - m_N 2^{N+1}.$$

As $N \to \infty$, the last term goes to 0 in $\mathbb{Z}_2$.

Proof idea

Putting the pieces together gives

$$\alpha = x + (q+1)\alpha - \sum_{i=1}^{r} q_i 2^i X_i + 2^r m_{r-1}.$$

Move the α terms to one side:

$$q\alpha = -x + \sum_{i=1}^{r} q_i 2^i X_i - 2^r m_{r-1}.$$

The right side is an ordinary integer. Call it p.
So

$$q\alpha = p, \qquad \text{and therefore} \qquad \alpha = \frac{p}{q}.$$

What happened?

The infinite tail collapsed to finite data: the starting bits and the carry.

Example: connection integer $q = 5$

If $q = 5$, then

$$q + 1 = 6 = 2 + 4,$$

so the rule is

$$\sigma_n = a_{n-1} + a_{n-2} + m_{n-1}.$$

Start with

$$a_0 = 1, \qquad a_1 = 1, \qquad m_1 = 0.$$

n	σ_n	new bit a_n	new memory m_n
2	$1 + 1 + 0 = 2$	0	1
3	$0 + 1 + 1 = 2$	0	1
4	$0 + 0 + 1 = 1$	1	0
5	$1 + 0 + 0 = 1$	1	0

So the sequence starts

$$11001100\cdots.$$

Example: its 2-adic value

The sequence

$$11001100\cdots$$

means

$$\alpha = 1 + 2 + 2^4 + 2^5 + 2^8 + 2^9 + \cdots .$$

Group the repeating blocks:

$$\alpha = (1 + 2)(1 + 2^4 + 2^8 + \cdots).$$

Since $2^4 = 16$,

$$\alpha = 3(1 + 16 + 16^2 + \cdots) = 3 \cdot \frac{1}{1 - 16} = -\frac{1}{5}.$$

This matches the theorem: the connection integer was 5.

What this means for cryptography

FCSRs can generate keystreams, but they are not magic.

The theorem says the output has structure:

$$\alpha = \frac{p}{q}.$$

If Eve sees the first N bits, she knows

$$\alpha \pmod{2^N}.$$

A 2-adic rational approximation attack asks:

$$\text{Can we find small } p, q \text{ with } \frac{p}{q} \equiv \alpha \pmod{2^N}?$$

Security issue

The carry makes the generator less linear, but the 2-adic structure can still guide attacks.

Takeaways

1. A bit string can be viewed as a 2-adic integer.
2. FCSRs are shift registers that remember carries.
3. The output of an FCSR is a rational 2-adic number p/q.
4. This gives both a construction idea and an attack idea.

Thanks for listening!